Reinforcement Learning: Rewards, Policies, and Q-Learning

Lecture 32

CS 4880 Kirk Duran

Learning Agents Improve Decisions from Outcomes

Unlike labeled examples, reinforcement learning supplies feedback after an action changes the world.



- An intelligent agent observes its environment, chooses actions, and receives feedback.
- The core cycle is **observe** $\rightarrow$ **decide** $\rightarrow$ **act** $\rightarrow$ **learn**.
- Neural networks can approximate perception, decisions, or action values when explicit tables become impractical.

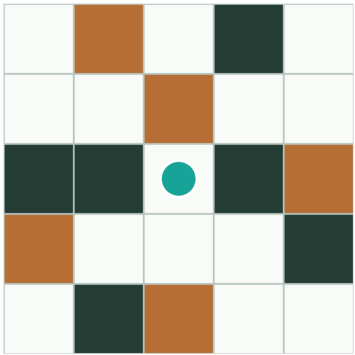
Key idea

We want a policy that performs well over repeated interactions, not just one correct prediction.

Our Running Example: A Vacuum Robot

Imagine a robot cleaning a room with obstacles and dirty tiles.

Original vacuum task: position + dirt pattern



- Dark cells are obstacles; brown cells are dirty; the teal circle is the robot.
- The robot knows its location, and movement is deterministic in this toy problem.
- It can move up, down, left, right, or clean.
- Search could plan every time; learning instead reuses experience.

What Exactly Is a State?

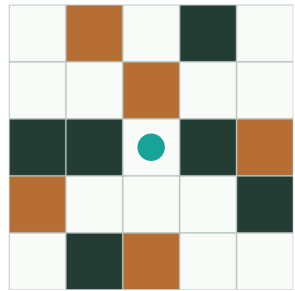
A state must distinguish situations that require different decisions.

- The vacuum state has the robot's **position** and the **clean/dirty status** of tiles.
- Two visits to the same square may be different states if the remaining dirt differs.
- A useful state representation captures information needed to predict the next state and reward.

Illustrative vacuum state

$$s = (\text{row}, \text{column}, \text{dirt configuration}).$$

Original vacuum task: position + dirt pattern

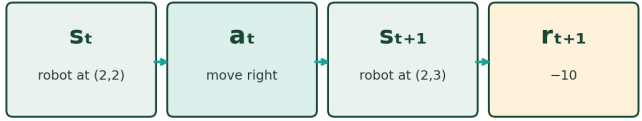


Key idea

The robot's position alone is not a complete state for the full cleaning task.

Actions Cause Transitions

The environment maps a current state and action into a next state and reward.



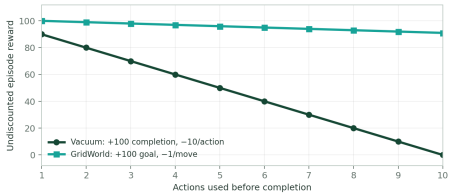
Known deterministic motion in the toy problem; reward follows the transition

- For the vacuum, actions are movement or cleaning; a move may be blocked by a wall.
- A **deterministic** transition always produces the same outcome from the same state-action pair.
- Real robots often have **stochastic** motion or incomplete information; we start with the easier case.

Reward Turns a Task Into an Optimization Problem

Not every successful behavior is equally efficient.

- **Original vacuum example:** $+100$ for cleaning the entire room and -10 for each action.
- Completing the task matters, but taking a long detour is costly.
- These are *designer-chosen* numbers, not quantities learned automatically.
- If reward is poorly designed, an agent may optimize the wrong behavior.



Key idea

Reward describes what the agent is encouraged to do; it is not a guarantee of sensible behavior.

Immediate Reward Is Not the Whole Objective

An action can be temporarily costly but lead to a better future.

Discounted return after time t

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}.$$

The discount factor $0 \le \gamma < 1$ reduces the weight of distant rewards in a continuing task.

- A direct route may cost fewer actions than a wandering route, even if both eventually finish.
- A policy seeks high **expected return**, not necessarily the largest immediate reward.
- For finite episodes, the sum stops at termination; $\gamma = 1$ can also be appropriate.

A Policy Is the Agent's Strategy

A policy says what the robot will do when it encounters a state.

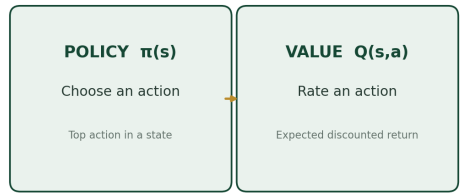
Deterministic policy

$$\pi(s) = a.$$

A mapping from each state s to one selected action a.

- A suboptimal policy wastes moves.
- A random policy ignores what the robot has learned.
- An optimal policy maximizes expected discounted return under the task model.

$\pi(s) = \text{argmax}_a\, Q(s,a)$



Key idea

Rather than plan from scratch every time, the agent can learn which action is best in each state.

Q-Learning Asks: How Good Is Each Action Here?

Q-learning estimates the value of choosing action a in state s and acting well afterward.

Optimal action-value function

$$Q^*(s,a) = \mathbb{E}[r_{t+1} + \gamma r_{t+2} + \cdots \mid s_t = s,\ a_t = a,\ \text{optimal continuation}]\,.$$

- A **Q-value** summarizes an immediate choice plus its possible consequences.
- The algorithm can learn from observed transitions without knowing the transition function in advance.
- After learning, the greedy policy selects the highest-valued action:

Greedy action choice

$$\pi(s) = \arg\max_{a \in A(s)} Q(s,a).$$

The Q-Table Is a State-by-Action Lookup Table

Rows represent states; columns represent actions; cells hold estimates of long-term value.

Before learning: all zeros

State	Up	Down	Left	Right
(0,0)	0	0	0	0
(0,1)	0	0	0	0
(1,0)	0	0	0	0
(1,1)	0	0	0	0

After learning: estimated values

State	Up	Down	Left	Right
(0,0)	12	**31**	9	25
(0,1)	16	**36**	22	29
(1,0)	18	**32**	11	28
(1,1)	21	**39**	20	34

Small illustrative table (not a measured Q-learning output)

- At initialization, entries are often all zero; experience updates the visited pairs.
- A row's largest entry determines a greedy action; illegal actions are excluded.

How One Experience Updates the Table

We move the current estimate toward a target built from reward plus the best next-state value.

Q-learning temporal-difference update

$$Q(s,a) \leftarrow Q(s,a) + \alpha \underbrace{\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]}_{\text{temporal-difference error}}.$$

1. Observe the current state s and choose action a.
2. Receive immediate reward r and new state s'.
3. Compute the **target** $r + \gamma \max_{a'} Q(s',a')$.
4. Move the old Q-value toward the target by fraction α.

Key idea

The update bootstraps: current estimates of future value improve earlier decisions.

Worked Example: Update $Q((2,2),\text{right})$

This reproduces the actual calculation from the old lecture.

- Current state $(2,2)$; action `right`.
- Next state $(2,3)$; reward $r = -10$.
- Old estimate $Q((2,2),\text{right}) = 5$.
- Best Q-value in the next state is 20.
- Learning rate $\alpha = 0.5$ and discount $\gamma = 0.9$.

Step 1: target

$$y = -10 + 0.9(20) = 8.$$

Step 2: update

$$Q_{\text{new}} = 5 + 0.5(8 - 5) = \boxed{6.5}.$$



The Learning Rate α and Discount Factor γ Do Different Jobs

Learning rate α

How far should a Q-estimate move toward newly observed evidence?

Small α: gradual adjustment.

Large α: larger, potentially noisy adjustment.

Discount factor γ

How much should future rewards influence decisions today?

$\gamma = 0$: immediate reward only.

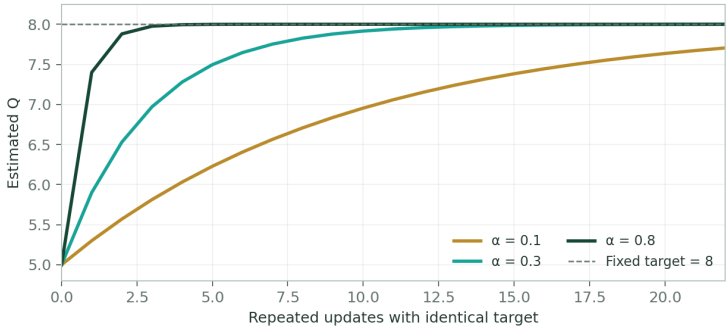
γ near 1: future rewards matter more.

Key idea

α controls the *update speed*; γ controls the *planning horizon*. Neither is the exploration probability.

A Controlled Illustration: What α Changes

Hold the update target fixed at 8 and start the estimated value at 5.



- With a *fixed* target, larger α moves more quickly toward it.
- In a real environment, the target also changes with experience; high α can amplify noise.

Exploration Versus Exploitation

Early Q-values are uncertain; always following the current best can prevent discovery.

Explore

Try an action to collect new experience. Some actions may initially look bad only because they have not been tested.

Exploit

Choose the highest-valued available action according to the current Q-table.

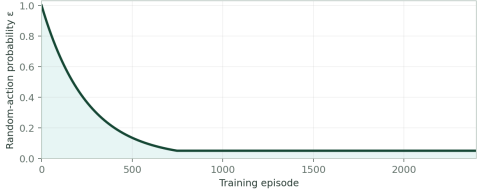
ε-greedy strategy

$$a = \begin{cases} \text{random legal action}, & \text{with probability } \varepsilon, \\ \arg\max_{a'} Q(s, a'), & \text{with probability } 1 - \varepsilon. \end{cases}$$

An ε-Greedy Action Rule in Python

Exploration is for training; demonstration or evaluation usually uses a greedy policy.

```
if rng.random() < epsilon:
    action = rng.choice(legal_actions)        # explore
else:
    values = Q[row, col, legal_actions]
    action = legal_actions[np.argmax(values)]  # exploit
```



- Example schedule: $\varepsilon_t = \max(0.05, 0.996^t)$ for episode t.
- Random choices must be limited to *legal* actions in our GridWorld implementation.

What Is an Episode?

One episode is one complete attempt between an initial state and a stopping condition.

1. Reset the environment to an initial state.
2. Observe s, select a, and take a step.
3. Receive the reward and next state; update $Q(s, a)$.
4. Repeat until reaching a goal, failure condition, or step limit.

Key idea

Learning generally takes many episodes. Episode return and task success help us tell whether behavior is improving.

Separate the Full Vacuum Task from the Teaching Demo

The original lecture moves from a rich cleaning task to a simpler position-only grid.

Vacuum robot (conceptual)	GridWorld (coded demo)
State: position *and* remaining dirt.	State: one of 25 positions.
Actions: move or clean.	Actions: up, down, left, right.
Reward: $+100$ when all cleaned; -10 per action.	Reward: $+100$ on reaching goal; -1 otherwise.
Obstacles and dirt are part of the task.	No obstacles or dirt; transitions deterministic.

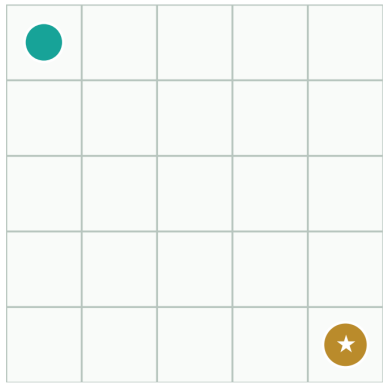
Key idea

Position-only Q-learning solves the smaller goal-reaching problem, not the entire vacuum-cleaning problem.

GridWorld Specification for a Reproducible Run

- A 5×5 grid; goal is bottom-right $(4, 4)$.
- Start each training episode at a random *non-goal* position.
- Every legal move is deterministic.
- Entering the goal gives $+100$ and ends the episode; all other moves give -1.
- Up to 100 moves per episode; a time limit is a truncation, not a goal.
- Parameters: $\alpha = 0.3$, $\gamma = 0.95$; 2400 episodes; seed 4880.

5 × 5 GridWorld: sample start and fixed goal



Demo Setup: Q-Values and GridWorld Dynamics

The Q-table has shape $5 \times 5 \times 4$ because each position has four candidate actions.

```
import numpy as np
rng = np.random.default_rng(4880)
Q = np.zeros((5, 5, 4))
ACTIONS = [(-1,0), (1,0), (0,-1), (0,1)]
GOAL = (4,4)

def step(state, action):
    row, col = state
    dr, dc = ACTIONS[action]
    nxt = (max(0, min(4, row+dr)),
           max(0, min(4, col+dc)))
    done = (nxt == GOAL)
    return nxt, (100 if done else -1), done
```

Key idea

The action-selection code filters illegal moves at boundaries.

Demo Core: Choose, Act, Update

One transition is enough for one Q-learning update.

```
legal = allowed_actions(state)
if rng.random() < epsilon:
    action = int(rng.choice(legal))
else:
    action = legal[np.argmax(Q[row, col, legal])]

next_state, reward, done = step(state, action)
future = 0 if done else max(
    Q[next_state[0], next_state[1],
      allowed_actions(next_state)])

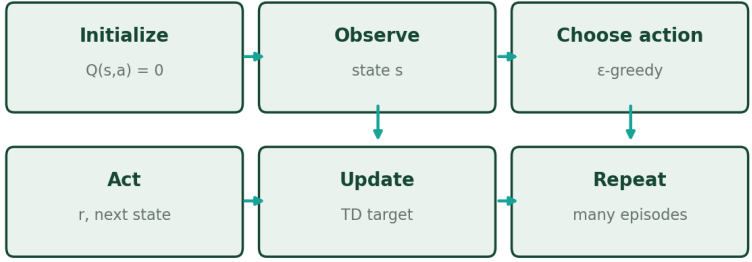
Q[row, col, action] += alpha * (
    reward + gamma * future - Q[row, col, action])
```

Key idea

For a terminal next state, the future-value contribution is zero.

The Complete Training Logic at a High Level

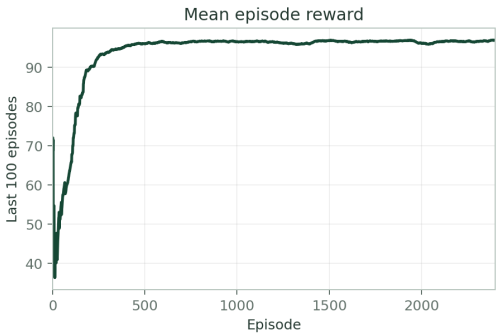
Repeat state transitions, but do not confuse training exploration with the final greedy policy.

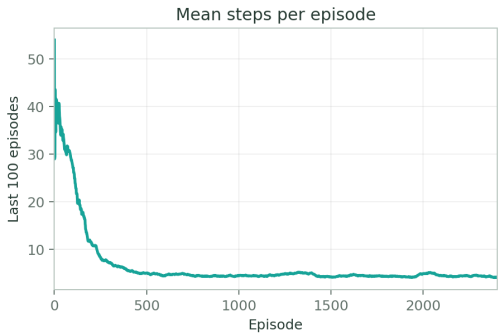


- Each episode resets to a new starting position and uses the current ε.
- After training, select the largest legal Q-value in each cell to draw the policy.

Demo Results: Learning Over 2400 Episodes

These supplied curves report a run using the stated seed and settings.

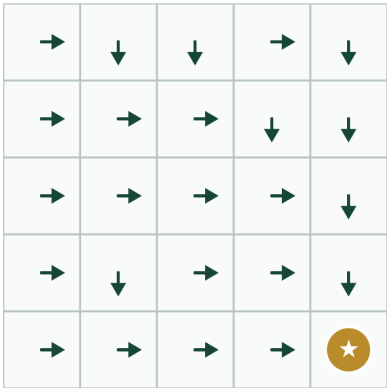




- Last 100 training episodes: **100%** goal success; mean reward **96.88**.
- These training outcomes include ε-greedy exploration.

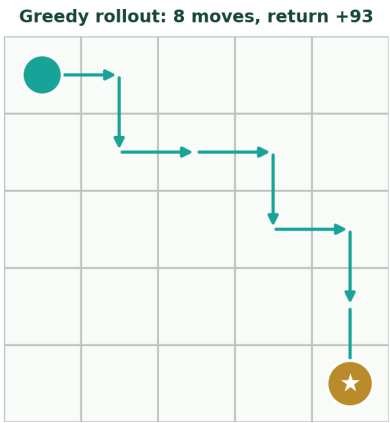
The Learned Policy: Select the Largest Legal Q-Value

Measured greedy policy after 2,400 episodes



- Arrows represent $\pi(s) = \arg\max_a Q(s, a)$ after training.
- The gold star is the terminal goal.
- Several shortest routes can be equally good, so an alternative seed may yield different arrows.
- This is the **simplified position-only GridWorld**.

Follow the Policy From the Top-Left Corner



- Beginning at $(0, 0)$, this learned greedy rollout reaches $(4, 4)$ in **8 moves**.
- Seven moves earn -1 each; the final goal-reaching move earns $+100$.
- Undiscounted episode return is $7(-1) + 100 = 93$.
- The training policy still sometimes explores; this rollout does not.

Why Not Merely Memorize the Path?

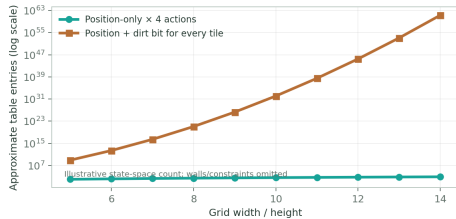
Q-learning stores a decision for *every* visited state, not only the one displayed route.

- Random training starts create experiences from many positions.
- The same learned table can guide the robot from alternative starting cells.
- Bootstrapped future values propagate information about the goal back to earlier positions.
- A grid has multiple shortest routes; the policy chooses among ties.
- This still does *not* mean the agent learned to recognize dirty tiles, plan around unseen obstacles, or clean the room.

Why Tabular Q-Learning Does Not Scale Well

The original lecture's final slide is not a minor footnote: state-space growth is the central limitation.

- One value is needed per **state-action pair**.
- A richer vacuum state must record position *and* remaining dirt.
- Learning can be slow: unvisited states receive no direct updates.
- Continuous sensor readings and images cannot be enumerated usefully.
- Random exploration may be costly or unsafe on real hardware.



From a Q-Table to a Learned Function

One natural continuation is to approximate Q-values from features instead of indexing a table.

Tabular Q-learning

$$Q[s, a]$$

One independent entry for each state-action pair. Works well in small, discrete problems.

Function approximation

$$Q_\theta(s, a)$$

A shared model (possibly a neural network) estimates values and can generalize across similar inputs.

Key idea

Function approximation helps with scale, but adds optimization and stability challenges; it is not a free fix.

Common Implementation and Evaluation Mistakes

1. **Mixing tasks:** vacuum-with-dirt and position-only GridWorld have different state spaces and rewards.
2. **Wrong terminal update:** do not add a next-state bootstrap at a true terminal state.
3. **Unclear boundaries:** decide whether invalid moves are blocked, penalized, or excluded.
4. **Only reporting one run:** a successful seed does not prove stability across seeds.
5. **Confusing train and test behavior:** exploratory training returns can differ from greedy evaluation.
6. **Treating a learning curve as a safety claim:** toy-grid success says little about a real robot.

Connecting Reinforcement Learning Back to Intelligent Agents

Reinforcement learning turns the agent loop into a method for improving behavior over time.

- **Environment:** supplies a new observation and reward after each action.
- **State:** summarizes the situation needed to choose an action.
- **Policy:** selects a behavior in each situation.
- **Q-values:** estimate the long-term consequences of choices.
- **Exploration:** gathers experiences needed to improve estimates.
- **Evaluation:** checks actual goal achievement, costs, and robustness.

Key idea

The central RL question is not "What is the correct label?" It is "Which behavior leads to the best outcomes over time?"

References, Provenance, and Reproducibility

- Source: *Artificial Intelligence, Lecture 22: Learning – Reinforcement Learning*, supplied 17-page slide deck, especially pages 2–17 (agent, vacuum grid, states, rewards, policies, Q-tables, update, worked example, ε-greedy, episodes, code, and limitations).
- Reproduced old-lecture worked result: $5 + 0.5[-10 + 0.9(20) - 5] = 6.5$.
- Demo extension: the supplied position-only 5×5 Q-learning experiment uses explicit legal actions, terminal handling, a fixed seed, and separate exploration/evaluation.
- Diagrams and reported GridWorld outcomes are included in the supplied image archive. The original figure-generation script was not included, so the reported training statistics have not been independently reproduced.
- Further reading: Sutton and Barto, *Reinforcement Learning: An Introduction* (2nd ed., 2018), Chapters 3 and 6; Watkins and Dayan (1992), *Q-learning*.
- Measured results may vary with changes to code, RNG, or numerical dependencies; the supplied run uses NumPy's `default_rng(4880)`.